

Programación <i>2º Evaluación</i>	
Tema 06. Estructuras de almacenamiento complejas	

RELACIÓN DE PROBLEMAS 11. Colecciones

Ejercicio 1

Escribir un programa para gestionar conjuntos de alumnos apuntados a equipos deportivos. El programa dispondrá de:

- La Clase *Alumno*, con los atributos: nombre y DNI de tipo String. Implementar el método constructor y los métodos toString() y equals.
- La Clase *Equipo*. Cada equipo deportivo se representa en una clase que contiene el nombre del equipo y un conjunto de alumnos.

El equipo tiene operaciones para:

- Añadir un alumno (recibe como parámetro el objeto alumno a insertar). Si el alumno ya existe en el equipo debe saltar una excepción.
- Borrar un alumno (recibe como parámetro el objeto alumno a borrar). Si el alumno no existe en el equipo debe saltar una excepción.
- Saber si un alumno pertenece al equipo. Recibe como parámetro el objeto alumno a buscar y devuelve null si no lo encuentra y el objeto alumno si existe.
- Mostrar en pantalla la lista de personas del equipo.
- Unión de equipos. El método lo llamará un equipo y se le pasará por parámetro el otro equipo con el que se quiere unir. Devuelve un equipo formado por los alumnos de ambos equipos.
- Intersección de equipos. Idem al anterior pero devuelve un equipo formado por los alumnos que están en los dos equipos

Se pide:

1. Decidir la estructura de almacenamiento más indicada para este ejercicio. No debe establecerse ningún límite de jugadores en los equipos.
2. Realizar las clases *Alumno*, *Equipo* así como un programa de prueba que cree varios equipos y pruebe todas las operaciones mostrando en pantalla los resultados.

Ejercicio 2

Modificar la clase anterior para que *Equipo* sea una clase genérica y se pueda aplicar a cualquier tipo.

Realizar dos programas principales, uno que pruebe la clase *Equipo* con la clase *Alumno* y otro que lo pruebe con la clase *Integer*.

Ejercicio 3

Realizar un programa que permita almacenar un historial de páginas web consultadas por un usuario. De cada página web se almacenará su url y la fecha y hora en la que se visitó.

Se debe realizar un menú de este tipo

1. Nueva página consultada: Se solicitará el nombre de la página y se tomará la fecha y hora del sistema insertándola en el historial. No se permitirá introducir una fecha y hora anterior a la última almacenada.
2. Consultar historial completo.
3. Consultar historial de un día.
4. Borrar historial completo.
5. Salir.

Se pide:

- Decidir la estructura de almacenamiento más indicada para este ejercicio. No debe establecerse ningún límite de entradas en el historial.
- Implementar la clase PaginaWeb, así como la clase Historial y el programa Principal.

NOTA: Utilizar la clase LocalDateTime para la fecha-hora.

Ejercicio 4

Se desea desarrollar una aplicación que permita enviar mensajes entre los profesores y los alumnos de un centro de enseñanza.

Se proporcionan la clase abstracta Persona, así como Profesor y Alumno. Se debe incluir las opciones para que las personas puedan enviarse mensajes. De cada mensaje se guardará el remitente, el texto del mensaje y la fecha y hora en la que se envió.

Todas las personas deben tener:

- Un método para poder enviar un mensaje a otra persona. Recibirá como parámetro la persona destinataria y el texto del mensaje.
 - Si la persona que envía el mensaje es un alumno y es menor de edad sólo puede enviar mensajes a profesores, es decir, si un alumno menor de edad intenta enviar un mensaje a otro alumno se debe provocar un error.
- Un método para poder leer los mensajes del buzón. Este método devolverá un String con todos los mensajes que tiene. Si no tiene mensajes para leer saltará el correspondiente mensaje de error. Cada mensaje irá precedido de un número 1 en adelante, es decir se mostrará con este formato.
"Mensaje X: De: nombreRemitente Texto: textoMensaje
Fecha y hora: DD-MM-AAAA HH:MM"
- Un método para poder leer los mensajes del buzón pero ordenado por el nombre del remitente alfabéticamente.

- Un método para poder borrar un mensaje del buzón. Al método se le pasará el número de mensaje que se desea borrar. Si no existe ese número de mensaje saltará una excepción.
- Un método que realice una búsqueda para poder encontrar los mensajes de su buzón que contienen una frase Este método devolverá un String con todos los mensajes que tienen esa frase o saltará la excepción si no encuentra ningún mensaje con esa frase.

Probar estos métodos desde el main.

Ejercicio 5

Crear un método genérico reverse que devuelva un ArrayList de objetos con la misma información, pero en orden inverso, es decir donde en la primera posición esté la información del último objeto, en la segunda la penúltima y así sucesivamente.

```
private static <T> ArrayList<T> reverse ( ArrayList<T> arrayOriginal)
```

Ejercicio 6

Realizar una clase que implemente un diccionario. Cada palabra puede tener varios significados. Se presentará un menú de esta forma

1. Añadir palabra. Se solicitará a palabra y su significado. Si la palabra ya tenía un significado se guardará este nuevo significado con los anteriores
2. Buscar palabra en diccionario: Se solicitará la palabra y se mostrarán todos sus significados
3. Borrar una palabra del diccionario: Se solicitará la palabra y se borrará, con todos sus significados.
4. Listado de palabras que empiecen por ... Se solicitará una cadena y se mostrará un listado de palabras ordenado alfabéticamente que comiencen por esa cadena.
5. Salir

NOTA: No debe considerarse ningún límite máximo de palabras. Decide el contenedor más adecuado teniendo en cuenta que la opción 2 será la más utilizada.

Ejercicio 7

En un gran almacén se tienen disponibles 20 cajas. Se desea implementar una aplicación para que cuando llegue un cliente se le indica a que caja debe acudir, la que tenga menos clientes esperando

Se debe desarrollar una aplicación con este menú:

1. **Abrir caja.** Se solicitará el número de caja y si no está abierta se abrirá. Si estuviese abierta se debe mostrar un mensaje de error
2. **Cerrar caja.** Se solicitará el número de caja. Sólo se puede cerrar, si estaba abierta y además no hay clientes, si no se mostrará el mensaje de error oportuno.
3. **Nuevo cliente:** Se generará un número automáticamente, de 1 en adelante, que identificará a este cliente. Se informará "Es usted el cliente número xx y debe ir a la caja número xx". El número de caja que se le asigna es la que tenga menos clientes esperando. En caso de empate debe ir a ser la caja de menor número.

4. **Atender cliente:** Se solicitará el número de caja en la que está y se mostrará un mensaje indicando "Se ha atendido al cliente con número XX". El cliente abandona la caja. Si no existen clientes en esta caja o bien estuviese cerrada se mostrarán los errores oportunos.
5. **Salir .**

Ejercicio 8

Se dispone de una serie de clases para la gestión de un Recetario. La clase Ingrediente tiene el nombre del ingrediente y la cantidad. La clase Receta tiene el nombre de la receta, su tiempo de preparación, un HashSet de ingredientes y un LinkedList de los pasos (String). El recetario contiene un HashMap de Recetas. Se pide:

- Clase Receta. Programar los los métodos siguientes:
 - 1.1. Método para saber si una receta necesita un ingrediente
`public boolean necesitaIngrediente(String nombreIngrediente)`
 - 1.2. Añadir un ingrediente a una receta. Si el ingrediente ya está sumará esta cantidad a lo que se necesita.
`public void annadirIngrediente(Ingrediente ingredienteNuevo)`
 - 1.3. Borrar un ingrediente de una receta. También debe borrar los pasos donde se nombre ese ingrediente.
`public void borrarIngrediente(Ingrediente ingredienteABorrar)
throws RecetaException`
 - 1.4. Método añadir un paso detrás de otro. Recibirá el texto del nuevo paso y el texto del paso detrás del cual se deberá insertar.
`public void annadirPasoDetrasDe(String pasoNuevo, String
pasoExistente) throws RecetaException`
- Clase Recetario. Programar los métodos siguientes
 - 1.1. Método para añadir una receta. Si ya existe una receta con ese nombre saltará la excepción.
`public void annadirReceta(Receta nuevaReceta) throws
RecetaException`
 - 2.2. Método que devuelva un String con un listado todas las recetas ordenadas alfabéticamente por nombre. Por cada receta aparecerá su nombre y el tiempo de preparación. Si no hay recetas debe saltar la excepción.
`public String listadoRecetasOrdenadasAlfabeticamente() throws
RecetaException`
 3. Método que devuelva un String con un listado de todas las recetas que contienen un ingrediente ordenadas de forma ascendente por el tiempo de preparación. Por cada receta aparecerá su nombre y el tiempo de preparación. Si no hay recetas con ese ingrediente saltará la excepción
`public String
listadoRecetasConIngredienteOrdenadasPorTiempoPreparacion(Stri
ng ingrediente) throws RecetaException`